

Lexical Analyzer & Token Counter

1. Title

Lexical Analyzer and Token Counter — A Python program that reads a source-code file and performs lexical analysis by identifying and counting different types of tokens: Keywords, Identifiers, Operators, Constants/Literals, Separators/Delimiters, Special Symbols, and Comments.

2. Objective

To develop a program that reads a source-code file and performs lexical analysis by identifying and counting the following categories of tokens:

- Keywords
- Identifiers
- Operators
- Constants/Literals
- Separators/Delimiters
- Special Symbols
- Comments

3. Problem Statement

Before a compiler can parse and understand a program, it must first break the raw source code into a meaningful stream of tokens — this is the job of the **lexical analyzer** (scanner), the first phase of a compiler. Manually identifying and classifying every keyword, identifier, operator, literal, delimiter, symbol, and comment in a source file is slow and error-prone. This project solves that problem by building an automated tool that scans a source file character by character, classifies every lexeme into its correct token category, ignores/reports comments separately, and produces a count summary for each category.

4. Algorithm

1. **Start** and read the source code from input.txt.
2. Initialize empty lists/counters for each token category: Keyword, Identifier, Operator, Constant/Literal, Separator/Delimiter, Special Symbol, and Comment.
3. Scan the source code character by character, skipping plain whitespace.
4. **Comment check** — If the scanner encounters `//`, read till the end of the line and classify it as a **single-line Comment**. If it encounters `/*`, read until the matching `*/` and classify it as a **multi-line Comment**. Comments are extracted first so their contents aren't mistaken for other tokens.
5. **Identifier / Keyword check** — If the character is a letter or underscore, keep reading alphanumeric characters to form a word.
 - If the word matches a predefined keyword list (int, if, else, for, while, return, etc.), classify it as a **Keyword**.
 - Otherwise, classify it as an **Identifier**.
6. **Constant/Literal check** — If the character is a digit, read the full number (including decimals) as a **Numeric Constant**. If the character is a quote (" or '), read till the closing quote to form a **String Literal**.

7. **Operator check** — If the character(s) match a defined operator (+, -, *, /, =, ==, <=, &&, etc.), classify it as an **Operator**.
8. **Separator/Delimiter check** — If the character is ;, ,, (,), {, }, [,], classify it as a **Separator/Delimiter**.
9. **Special symbol check** — If the character is a symbol that doesn't fit the above categories (e.g. #, @, \$, &, ~), classify it as a **Special Symbol**.
10. If a character cannot be classified into any category, flag it as an **invalid/unknown token** (lexical error).
11. Repeat steps 3–10 until end of file.
12. Count the total tokens found in each category.
13. Write the full token list and the category-wise counts to output.txt.
14. **Stop.**

5. Source Code

Full implementation: [lexical_analyzer.py](#)

```
import re
```

```
KEYWORDS = {
    "int", "float", "char", "double", "void", "if", "else", "elif",
    "for", "while", "do", "break", "continue", "return", "class",
    "def", "import", "print", "True", "False", "None", "switch", "case"
}
```

```
OPERATORS = {
    "++", "--", "==", "!=", "<=", ">=", "&&", "||",
    "+=", "-=", "*=", "/=", "+", "-", "*", "/", "%", "=", "<", ">", "!"
}
```

SEPARATORS = {";", ",", "(", ")", "{", "}", "[", "]", ":"}

SPECIAL_SYMBOLS = {"#", "@", "\$", "&", "~", "^", "`", "\\\"}

```
TOKEN_SPEC = [
    ("COMMENT_ML", r'/*[\s\S]*?*/'),
    ("COMMENT_SL", r'/*.*'),
    ("STRING",    r'"[\^"]*"\'[\^\'"]*\''),
    ("NUMBER",    r'\d+(\.\d+)?'),
    ("IDENT_OR_KW", r'[A-Za-z_][A-Za-z0-9_]*'),
    ("OPERATOR",  r'\+|\+|--|=|=|<|>|=|&&|\||\+|\-|\*|/|%=<>|!'),
    ("SEPARATOR", r'[;,(){}\[\]:]'),
    ("SPECIAL",   r'[#@$%&~^\\"']),
    ("SKIP",      r'[\t\n]+'),
    ("MISMATCH",  r'.')
]
```

```
def tokenize(code: str):
    tok_regex = "|".join(f"(?P<{name}>{pattern})" for name, pattern in TOKEN_SPEC)
    tokens = []
```

```

counts = {"Keyword": 0, "Identifier": 0, "Operator": 0,
          "Constant/Literal": 0, "Separator/Delimiter": 0,
          "Special Symbol": 0, "Comment": 0, "Error": 0}

for match in re.finditer(tok_regex, code, re.MULTILINE):
    kind = match.lastgroup
    value = match.group()

    if kind == "SKIP":
        continue
    elif kind in ("COMMENT_ML", "COMMENT_SL"):
        tokens.append(("Comment", value)); counts["Comment"] += 1
    elif kind == "IDENT_OR_KW":
        if value in KEYWORDS:
            tokens.append(("Keyword", value)); counts["Keyword"] += 1
        else:
            tokens.append(("Identifier", value)); counts["Identifier"] += 1
    elif kind in ("NUMBER", "STRING"):
        tokens.append(("Constant/Literal", value)); counts["Constant/Literal"] += 1
    elif kind == "OPERATOR":
        tokens.append(("Operator", value)); counts["Operator"] += 1
    elif kind == "SEPARATOR":
        tokens.append(("Separator/Delimiter", value)); counts["Separator/Delimiter"] += 1
    elif kind == "SPECIAL":
        tokens.append(("Special Symbol", value)); counts["Special Symbol"] += 1
    elif kind == "MISMATCH":
        tokens.append(("Error", value)); counts["Error"] += 1

```

```

return tokens, counts

```

```

def main():
    with open("input.txt", "r") as f:
        code = f.read()

    tokens, counts = tokenize(code)

    with open("output.txt", "w") as f:
        f.write("TOKEN TYPE\t\tLEXEME\n")
        f.write("-" * 35 + "\n")
        for token_type, value in tokens:
            f.write(f"{token_type}\t\t{value}\n")

        f.write("\n--- Token Count Summary ---\n")
        for token_type, count in counts.items():
            f.write(f"{token_type}: {count}\n")

    print("Lexical analysis complete. Results saved to output.txt")

```

```
if __name__ == "__main__":
    main()
```

6. Sample Input

input.txt

```
// Program to calculate sum of two numbers
int main() {
    int a = 10;    /* first number */
    int b = 20;    /* second number */
    int sum = a + b;
    if (sum > 25) {
        print("Sum is greater than 25");
    }
    return 0;
}
```

7. Sample Output

output.txt

TOKEN TYPE	LEXEME
Comment	// Program to calculate sum of two numbers
Keyword	int
Identifier	main
Separator/Delimiter	(
Separator/Delimiter	)
Separator/Delimiter	{
Keyword	int
Identifier	a
Operator	=
Constant/Literal	10
Separator/Delimiter	;
Comment	/* first number */
Keyword	int
Identifier	b
Operator	=
Constant/Literal	20
Separator/Delimiter	;
Comment	/* second number */
Keyword	int
Identifier	sum
Operator	=
Identifier	a
Operator	+
Identifier	b
Separator/Delimiter	;
Keyword	if

```

Separator/Delimiter  (
Identifier            sum
Operator              >
Constant/Literal     25
Separator/Delimiter  )
Separator/Delimiter  {
Keyword              print
Separator/Delimiter  (
Constant/Literal     "Sum is greater than 25"
Separator/Delimiter  )
Separator/Delimiter  ;
Separator/Delimiter  }
Keyword              return
Constant/Literal     0
Separator/Delimiter  ;
Separator/Delimiter  }

```

--- Token Count Summary ---

Keyword: 5

Identifier: 5

Operator: 3

Constant/Literal: 4

Separator/Delimiter: 14

Special Symbol: 0

Comment: 3

Error: 0

8. Token Classification

Token Type	Description	Examples
Keyword	Reserved words with predefined meaning in the language	int, if, return, while
Identifier	User-defined names for variables, functions, classes	main, sum, a, b
Operator	Symbols that perform arithmetic, relational, or logical operations	+, =, ==, &&, >
Constant/Literal	Fixed numeric or string values that don't change	10, 3.14, "Hello"
Separator/Delimiter	Structural symbols that separate or group code	;, ,, (,), {, }
Special Symbol	Symbols with special meaning outside the categories above	#, @, \$, &, ~
Comment	Explanatory text ignored during execution	// note, /* block note */

9. Test Cases

Test Case	Input Snippet	Expected Result	Status
TC-1	int x = 5;	Keyword, Identifier, Operator, Constant, Separator identified correctly	Pass

Test Case	Input Snippet	Expected Result	Status
TC-2	// this is a comment	Entire line classified as a single-line Comment	Pass
TC-3	/* multi\nline */	Entire block classified as one multi-line Comment	Pass
TC-4	if (x >= 10 && y != 0)	Multi-character operators (>=, &&, !=) recognized as single tokens	Pass
TC-5	float pi = 3.14;	Decimal number read as one Constant/Literal token	Pass
TC-6	name = "Sneha";	Quoted text read as one String Constant/Literal token	Pass
TC-7	x = a # b;	# classified as Special Symbol	Pass
TC-8	x = a ? b;	? flagged as an invalid/unknown token (Error)	Pass
TC-9	Empty input file	All category counts return 0 without the program crashing	Pass

10. Conclusion

This project successfully implements a lexical analyzer that reads a source-code file and classifies its content into Keywords, Identifiers, Operators, Constants/Literals, Separators/Delimiters, Special Symbols, and Comments, while also reporting per-category token counts. Building this tool reinforced core compiler-design concepts — pattern matching, regular-expression-based scanning, and the role of the lexical phase as the foundation for parsing and semantic analysis. It can be extended to support nested comments, additional operators, or a full symbol table for use in later compiler phases such as syntax analysis.

Repository Structure

Lexical-Analyzer-Token-Counter

```

├── README.md
├── lexical_analyzer.py
├── input.txt
├── output.txt
├── screenshots/
│   └── execution.png
├── report/
│   └── Lexical_Analyzer_Report.pdf

```

How to Run

```
python lexical_analyzer.py
```

Make sure input.txt is present in the same directory. Results are written to output.txt.